

Imperial College London
Department of Earth Science and Engineering
MSc in Environmental Data Science and Machine Learning

Independent Research Project
Final Report

VHR-Seg: A Coarse-to-Fine Deep Learning Framework for Semantic Segmentation in Very High Resolution Remote Sensing Imagery

by

Yaowen Chang

Email: yc5224@imperial.ac.uk
GitHub username: esemsc-yc5224
Repository: <https://github.com/ese-ada-lovelace-2024/irp-yc5224>

Supervisors:

Dr. Yves Plancherel
Dr. Myriam Prasow-Emond

August 2025

Abstract

Very high-resolution (VHR) remote sensing image semantic segmentation holds great potential for applications in ecological conservation, urban management, and many other domains. However, the large scale of VHR imagery, the high annotation cost, and the lack of standardized workflows present major challenges for semantic segmentation. As a result, most existing deep learning approaches are restricted to specific datasets, which limits their cross-domain applicability.

To address these challenges, a two-stage coarse-to-fine segmentation framework, VHR-Seg, is proposed. The framework provides end-to-end functionalities including automated geospatial data processing, model training and inference, and annotation guidance. Segmentation can be performed according to user-defined regions of interest. The designed High-Resolution (HR) series models embedded in the framework effectively capture fine details while preserving contextual semantic information, enabling fine-grained segmentation of VHR imagery.

To evaluate the performance of VHR-Seg, comparative experiments were conducted between HR-series models, baseline approaches, and state-of-the-art (SOTA) methods under the same experimental settings. Results demonstrate the efficiency of our segmentation models. Exploratory discussions on image resolution and training dataset scale reveal the performance boundaries of fine-grained segmentation across different categories. The potential of leveraging the distinct characteristics of each stage in VHR-Seg to enhance annotation efficiency and accuracy has also been investigated.

1 Introduction

Satellite and airborne remote sensing is currently undergoing a technical revolution with the appearance and blooming development of very high-resolution sensors. Benefiting from advances in sensor technologies, optical systems, and data processing algorithms, the spatial resolution of both satellite and unmanned aerial vehicle (UAV) imagery has improved dramatically in recent years [10], reaching sub-meter levels for commercial satellites [20, 36] and centimeter levels for UAVs [28, 30]. Such improvements in VHR RSIs have greatly expanded application potential. Researchers apply VHR remote sensing images (RSIs) to map features as small as insects and other fauna with centimeter-level precision over vast areas. These breakthrough achievements make high-frequency biodiversity monitoring possible and provide critical data for tracking endangered species and their recovery plans. In addition to these applications, VHR urban remote sensing data is capable of generating accurate urban functional area distribution maps [21], which is of great significance to improve urban governance capacity and achieve the sustainable development goals [37].

The growing demands to utilize VHR RSIs call for new data processing methods [44]. In recent years, a large number of researchers have adopted machine learning (ML) and deep learning (DL) methods to analyze RSIs [1]. Among the various tasks, semantic segmentation has emerged as one of the most crucial, as it focuses on the precise delineation of semantic regions within remote sensing imagery [5, 42]. The advent of deep learning has greatly promoted this field and led to the development of numerous effective algorithms.

Early research focused on designing CNN-based architectures [43] to capture local features and hierarchical representations. However, CNN-based models remain constrained in modeling global dependencies, which has motivated the adaptation of Transformer architectures [38] for RSIs semantic segmentation. UNetFormer [39] constructs the Transformer-based decoder and further proposes an UNet-like Transformer (UNetFormer) for real-time remote sensing segmentation. More recently, foundation models have appeared, aiming to generalize across diverse

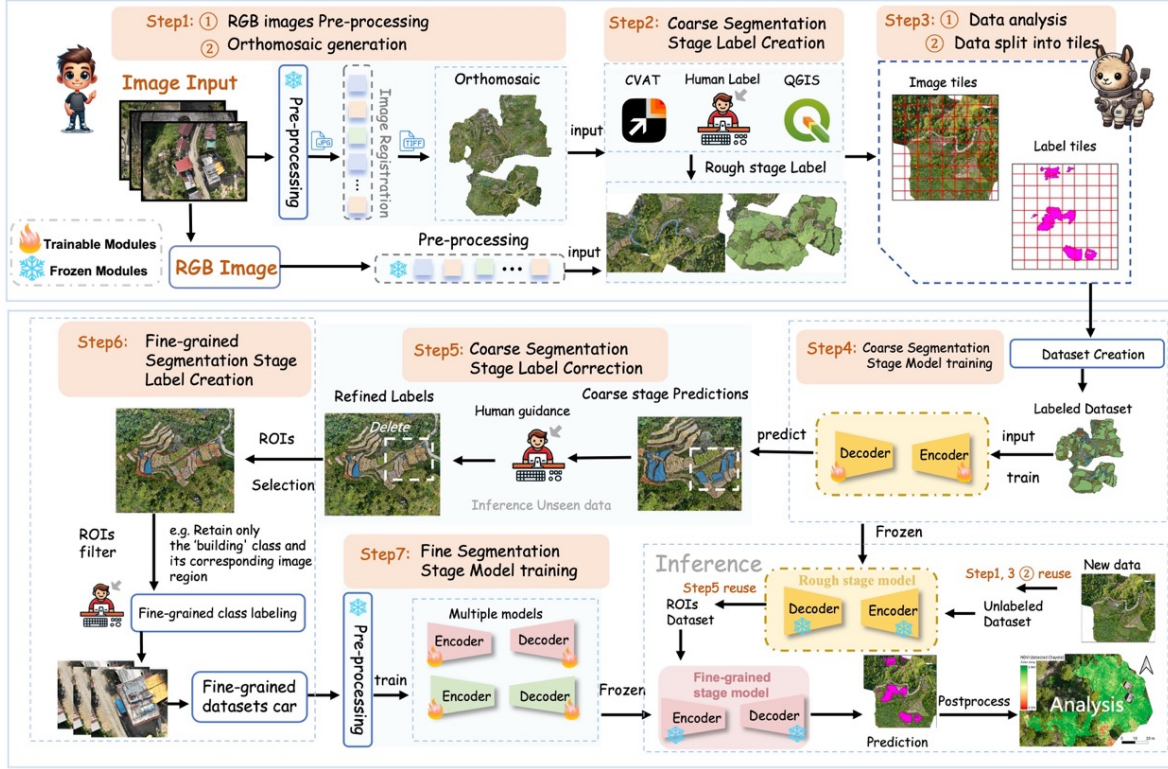


Figure 1: Overall framework of VHR-Seg. The framework consists of seven main steps. Model training, data preprocessing, and data postprocessing are automated, while label creation, correction, and ROI selection rely on human expertise.

domains without task-specific retraining [11]. Segment Anything introduced a segmentation foundation model (SAM) for promptable segmentation in images [24]. It is a zero-shot segmentation model developed by Meta. As the name suggests, the model is intended for segmenting arbitrary scenes without requiring additional tuning. SAM2 [34] is the second generation of the SAM, designed for instance-level segmentation in both image and video tasks. Designed to cover as many use cases as possible, SAM2 is a robust solution for segmenting commonly occurring features. The model can either be used interactively by clicking on image regions to include or exclude, or it can segment a scene autonomously. However, despite their strong generalization in natural images, foundation models such as SAM and SAM2 still struggle in VHR RSIs due to their unique characteristics.

While many semantic segmentation methods have achieved great success in the domain of natural images, they often fail to achieve comparable performance when directly migrated to VHR RSIs. There exist three challenges in the semantics segmentation in VHR RSIs.

Spatial size difference. VHR RSIs are usually much larger in size than typical natural images used in computer vision applications. For example, images in the PASCAL VOC data set [18] (an open benchmark for semantic segmentation of natural images) have a size of 500×375 pixels, whereas images in the ISPRS semantic labeling data set (Potsdam area) [16] have 6000×6000 pixels and the original images in the Beijing land-use classification data set have 47244×47244 pixels. The significant difference in the spatial size makes it impossible to give a whole VHR RSI as input to semantic segmentation models; otherwise, the computational costs would be unaffordable [26].

Lack of standardization and reproducibility. Current remote sensing (RS) research in very high resolution (VHR) mapping faces a major challenge due to the inconsistency in deep learn-

ing (DL) model design and usage. Many studies develop custom models tailored to specific datasets or features [1, 29], and literature reviews have revealed a notable lack of standardized DL workflows [23, 19, 31]. This hinders the transferability of DL implementations, especially as VHR RS datasets are diverse and feature-rich. It becomes infeasible to build new models and workflows from scratch for each new feature studied.

Scarcity of annotated data poses a major obstacle to the advancement of VHR RSIs semantic segmentation.

To address these challenges, a coarse-to-fine deep learning framework for semantic segmentation in very high resolution remote sensing imagery is proposed. The objectives of this research are listed as follows : (i) design a scalable framework that integrates annotation, preprocessing, training, and inference into a reproducible pipeline; (ii) reduce computational and annotation costs by regions of interests (ROIs) selection in fine-grained segmentation stage; (iii) develop HR-series models capable of preserving contextual semantics while accurately segmenting fine-scale objects; and (iv) conduct systematic experiments on UAV-based datasets to evaluate the effects of image resolution and dataset scale on segmentation performance. Through these objectives, the study aims to provide a practical, efficient, and reproducible solution for semantic segmentation of VHR remote sensing imagery.

2 Methodology

2.1 Study area and Data

The dataset to be used in this project is a series of 5280 x 3956 RGB images and an RGB orthomosaic formed from these UAV images. The image was mapped over 2 km² of the legacy mining site of Sto. Niño in Tublay, Benguet province, the Philippines. The overview of the dataset is provided in the Appendix B. A DJI Mavic 3 MS drone equipped with a Zenmuse L1 lidar/photo camera was flown at 50m altitude to capture this data in November 2023. The spatial resolution is 2-3 cm/px.

2.2 Dataset Annotation

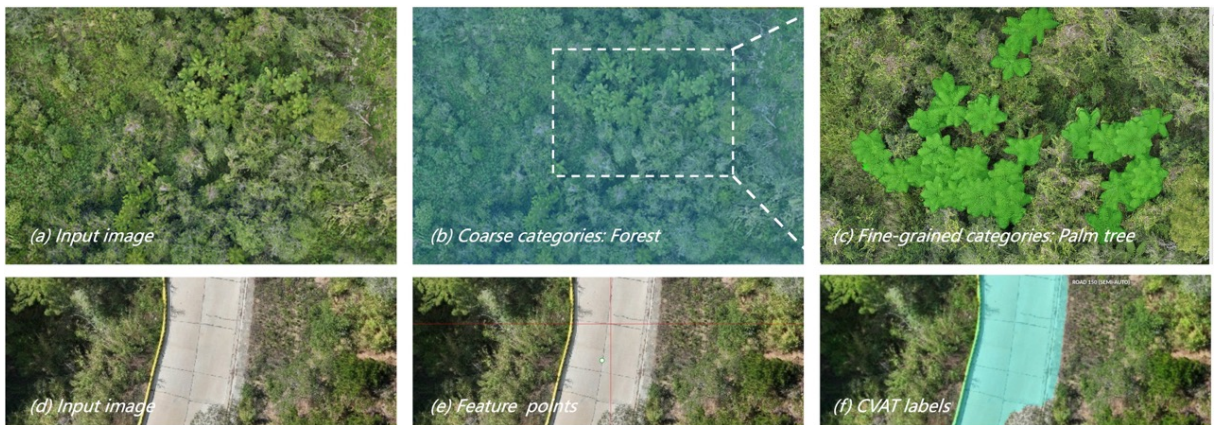


Figure 2: Labeling workflow for the Sto. Niño dataset. First row: QGIS-based annotation. (a) Palm plantation region; (b) coarse category label "forest" with blue mask; (c) refinement to fine-grained category "palm tree" (green). Second row: CVAT-based annotation. (d) Input image; (e) user-placed points indicating target objects; (f) automatically generated labels.

A two-stage coarse-to-fine annotation strategy was employed to balance annotation efficiency with segmentation quality. The dataset was annotated using QGIS and CVAT. CVAT supports the use of models to speed up annotation. The workflow is shown in Figure 2.

This two stage process allows both broad coarse-level annotations and subsequent fine-grained annotations to be incorporated seamlessly. In the first stage, a limited portion of the imagery was annotated into coarse land-cover categories, including forest, cropland, grassland, building, road, and bare soil. These broad labels can be generated relatively quickly because annotators are only required to discriminate major landscape types rather than object-level boundaries. A preliminary coarse segmentation model is then trained and applied across the entire dataset to generate semantic masks. Based on these masks, users can select regions of interest (ROIs). For example, if palm trees are of interest, annotation is restricted to forest areas. In the second stage, fine-grained labels are created only inside the selected ROIs. For example, inside the region of forest, palm trees and banana trees will be annotated and will ignore the irrelevant regions. By restricting fine annotation to these ROIs, the amount of detailed labels on VHR RSIs is significantly reduced.

This hierarchical annotation workflow reduces annotation workload while enabling flexible and incremental extension when new areas or target classes are introduced. To ensure reproducibility, documentation describing the labeling procedure in QGIS is provided in Appendix C. In this project, Forest, Road, and Building were selected as coarse categories, whereas Palm tree, Banana tree, and Car (Appendix D) served as fine-grained categories for segmentation.

2.3 VHR-Seg: An Open-Source Pipeline for Coarse-to-Fine Segmentation

VHR-Seg is a platform-independent command-line tool implemented in Python 3.10. The system is modular and designed for practical geospatial deployments. The repository includes step-by-step setup and usage notes, and the documentation is auto-generated to HTML with Sphinx. To ensure reproducibility across machines, all dependencies were managed using `pip-tools`, which compiles a reproducible `requirements.txt` file from a lightweight specification. Users can therefore install the full environment on new machines with a single command. To avoid complications with GDAL's OS-specific dependencies, Anaconda was used to manage the library's integration. VHR-Seg's deep learning modules are implemented with the MM-Segmentation framework, enabling flexible configuration for coarse-grained and fine-grained models. All components rely exclusively on open-source software.

2.3.1 System Architecture

As shown in Figure1, the VHR-Seg pipeline consists of seven stages, which can be grouped into three main blocks: preprocessing, training and evaluation, and the postprocessing of predictions into final segmentation outputs. To design the code, VHR-Seg use 'workflow' to represent these steps. A UML diagram illustrating the VHR-Seg workflows is provided in the Appendix E, where the interconnections between different workflows are described in detail. For clarity, we also refer to a workflow as a "task." VHR-Seg supports both the execution of individual tasks and running the entire pipeline in a single process. To ensure efficiency, a multithreaded design pattern has been adopted. Further implementation details are discussed in the Appendix F. The CLI allows users to selectively run any stage, or to continuously update only the fine stage as new ROIs or subclasses become available. Internally, multiprocessing is supported to accelerate large-scale inference over thousands of tiles, and the system works with common geospatial raster formats (GeoTIFF, PNG, JP2, etc.) and polygon label files (Shapefiles, COCO JSON, CVAT XML). A complete list of available Tasks and their dependencies is provided in Appendix F.

2.3.2 Data Preprocessing.

In VHR-Seg workflows, two types of imagery are typically used as input: raw UAV-captured images that have not been preprocessed into orthophotos, and georeferenced orthophotos or other large-scale geospatial products. While deep learning models usually process relatively small image patches (e.g., 512×512 pixels), geospatial data can be orders of magnitude larger. A common approach to address this issue is downsampling. However, in VHR RSIs semantic segmentation task, the key advantage lies in preserving fine-grained details, and thus aggressive downsampling may not be an appropriate choice. Alternatively, splitting large RS imagery into tiles is also challenging: orthomosaics often exceed the memory capacity of standard GPUs, and tiling can introduce boundary artifacts because adjacent patches are processed independently. To overcome these challenges, VHR-Seg processes input images through memory mapping rather than loading them fully into memory.

To maintain overlap across adjacent tiles, a sliding-window approach is adopted. This strategy improves the consistency of predictions and yields roughly $(1/s)^2$ times more training samples, where $0 < s < 1$ denotes the stride ($s = 0.5$ in our project experiments). Output image and label tiles are saved as compressed JPG files or PNG files (lossless), respectively. The georeferencing from the original input is preserved and reapplied, allowing the final segmentation maps to be correctly aligned when re-imported into GIS software.

2.3.3 Utilizing MMSegmentation

Recently, an increasing number of well-maintained open source projects [4, 8, 14] have adopted MMsegmentation as a foundation for developing custom models and tools. The advantages of MMsegmentation framework include active maintenance, support for a wide range of state-of-the-art (SOTA) semantic segmentation methods [13], high efficiency, and strong extensibility. The core of MMSegmentation lies in highly abstracted, simple configuration files that automatically generate PyTorch models. Compared to directly using PyTorch or other deep learning libraries, using the MMsegmentation framework has proven to be both effective and sustainable. Given these advantages, VHR-Seg adopts MMSegmentation as its backbone framework, providing flexible support for model training and inference within the hierarchical segmentation pipeline. The more diverse the tiles, the less likely the model is to overfit, thereby improving its generalization ability.

Training methods. Semantic segmentation datasets often suffer from severe class imbalance, a problem also observed in the Sto. Niño dataset. In order to mitigate the imbalance between majority classes and minority classes. A class-balance algorithm was proposed, as shown in Appendix G.1. The weights will be given to every images from training set, which is a kind of oversampling. Although oversampling may increase the model performance, it may introduce redundancy, causing model overfitting. To deal with the potential problem, randomized image augmentations are applied by using Albumentations library [3]. Augmentations like downscaling and color shifts increase tile diversity on the training set [25, 35]. The more diverse the tiles, the less likely the model is to overfit, thereby improving its generalization ability.

Dataset splitting. VHR-Seg allows users to create customized datasets by selecting their regions of interest (ROIs). Besides, It also provides an automatic mode that randomly splits the dataset. However, remote sensing data are typically spatially auto-correlate[11]. Therefore, dividing datasets based on users' interest can help reduce spatial leakage and mitigate model overfitting.

2.3.4 Postprocessing

Since the model operates on partially overlapping tiles, pixel-wise predictions must be merged back into a seamless full-resolution segmentation map. The merging strategies have a direct impact on segmentation map visual quality and consistency. Two merging strategies are supported in our pipeline. (1) logit-merge, which fuses overlapping regions by weighting them with the model’s confidence scores. (2) crop-merge, which discards boundary areas of tiles and keeps only their centers, except near image edges. The crop-based approach generally produces more accurate results while also being computationally faster. The detail information is shown in Appendix G

2.4 Model design and Selection

As discussed in Section 1, VHR semantic segmentation typically requires substantial computational resources, and existing pre-trained models (e.g. SegFormer [41] and Mask2Former [7]) often perform poorly in fine-grained segmentation tasks involving highly similar categories, such as palm trees and banana trees. Therefore, it is worth investigating how to improve fine-grained segmentation performance while preserving the advantages of VHR RSIs under controllable computational costs. Inspired by HRDA [22], SegFormer and Mask2Former, both of which have achieved state-of-the-art (SOTA) performance [11] on natural image benchmarks, are adopted as the backbone networks, and combined with HRDA dual resolution encoder - decoder head [22]. This leads to the proposed HRsegFormer and HRmask2Former, which achieve superior segmentation performance on fine-grained targets. The detail model design is illustrated in Appendix J.

To comprehensively evaluate the performance of our proposed model, UNet [15] was selected as baseline model. In order to prove the effectiveness of HR-series model, the ablation study was conducted in Appendix K

2.5 Experiment

2.5.1 Experiment Design

A comprehensive evaluation of VHR-Seg considered three key survey factors: object scale, Ground Sample Distance (GSD), and the amount of training data. To examine the role of feature scale, models were trained separately on outputs from the coarse and fine-grained stages of our framework, which capture objects of markedly different sizes and spectral patterns.

To assess sensitivity to spatial resolution, we trained models for each feature type on twelve dataset versions with progressively coarser GSDs. Nine of these correspond to VHR satellite sensors, enabling us to test the applicability of satellite imagery for VHR-focused tasks. The full set of GSDs is listed in Appendix H, with 0.08 m/px as the upper limit due to computational constraints. Resolution was degraded using several synthetic downsampling strategies (Fig. 3), detailed in Appendix I. Importantly, method B reduced only the image resolution, leaving annotations intact. Considering multiple degradation schemes provides a more reliable view of performance trends, reducing bias from any single method.

Finally, how training set size influences model accuracy by progressively reducing the number of images available for training was studied. 120 models in two-stage segmentation tasks were trained under these different conditions.

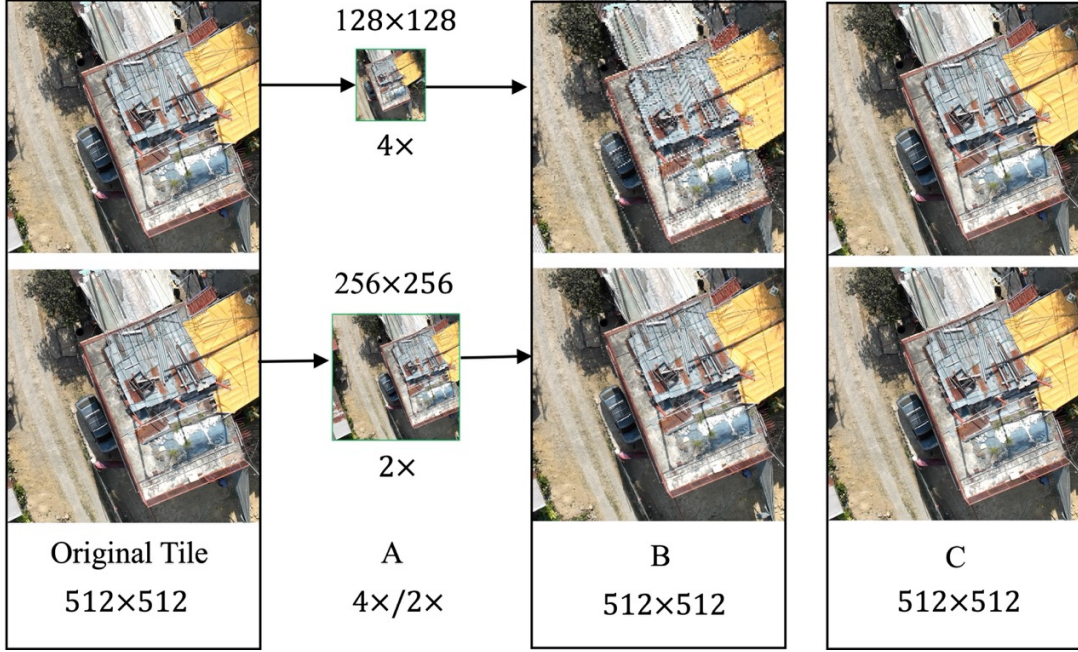


Figure 3: Different approaches for orthomosaic tiles (A–C) resolution reduction and their impact on upsampling recovery

2.5.2 Experiment Setup

Models were trained on the high-performance computing (HPC) platform of Imperial College London in both two-stage, using a single NVIDIA A100 GPU (40 GB VRAM), 32 GB of system memory, and 4 Intel CPU cores. To support reproducibility, same random seed was applied across all libraries, with consistent software versions maintained (CUDA 12.1, PyTorch 2.1.0+cu121, and TorchVision 0.16.0+cu121). Although exact bitwise reproducibility across hardware cannot be guaranteed, these settings help to minimize variability. Fixed batch size and random seeds ensure that training samples are presented in a consistent order across runs, regardless of the hardware environment.

To reduce spatial leakage and maintain balanced class distributions, the orthomosaic imagery and corresponding labels were manually split into 70% training, 10% validation, and 20% test subsets (Fig. 4). The same ratio was applied in the fine-grained stage for the selected ROIs. Mask2Former [7] was used as the coarse-stage segmentation model. The model was configured with a Swin-S backbone [17] pretrained on the Cityscapes dataset [12]. The backbone networks were kept unfrozen to enhance transfer learning capacity. In the fine-grained stage, HRsegFormer and HRmask2Former were employed to segment the detail classes, using SegFormer-B2 and Swin-S backbones, respectively.

For all stages, models were trained with an initial learning rate of 6×10^{-5} , which is scheduled by the poly strategy with power 0.9 over 200 epochs. The optimizer is AdamW [27]. with epsilon $= 10^{-8}$, weight decay 10^{-4} , and a batch size of 4 on each GPU. No hyperparameter tuning was needed in the coarse stage, while the downsampling factor s and relative crop size from HR-series model need were analyzed. The ablation study in the Appendix K shows that the model is robust to s , and a suitable value was found to be 2.

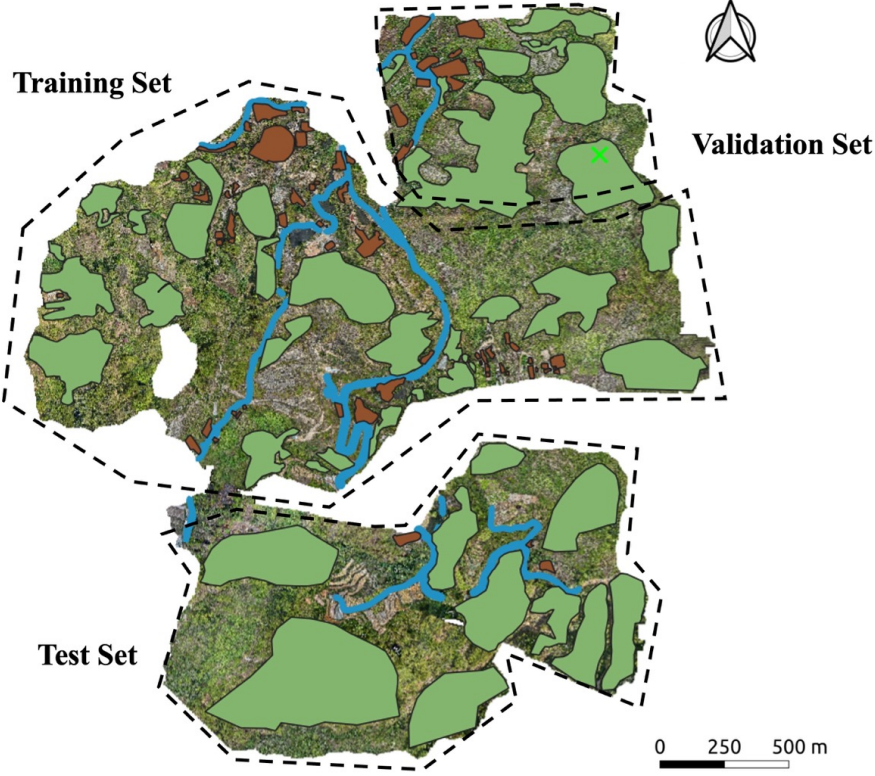


Figure 4: Dataset split of the Sto. Niño imagery into training, validation, and test regions in coarse stage. Ground truth annotations are shown, containing a balanced representation of classes. A buffer zone was added to separate training from evaluation regions, reducing spatial leakage.

2.5.3 Evaluation Metric

Following standard evaluation protocols from prior methods [9, 40], The intersection over union (IoU), Dice coefficient (F1 score) metrics and mean intersection over union(mIoU) were used for model evaluation. IoU evaluates segmentation accuracy by comparing the area of overlap to the union between predictions and ground truth [11]. The Dice coefficient is the harmonic mean of precision and recall: Precision describes the fraction of positive predictions that are correct, recall gives the proportion of positive samples that the model correctly predicted as positive. The mIoU is computed as the average IoU across all classes:

$$\text{IoU}_i = \frac{TP_i}{TP_i + FP_i + FN_i}, \quad \text{mIoU} = \frac{1}{N} \sum_{i=1}^N \text{IoU}_i, \quad (1)$$

where TP_i , FP_i , and FN_i represent true positives, false positives, and false negatives for class i , N is the total number of classes. The Dice coefficient is computed as following equations:

$$\text{Dice coefficient}_i = \frac{2TP_i}{2TP_i + FP_i + FN_i}, \quad (2)$$

Besides, models complexity and inference speed are measured in terms of FLOPs and FPS. The demand of memory usage is characterized by the number of parameters (P) and the overall memory footprint.

3 Results

3.1 Profiling of VHR-Seg Performance

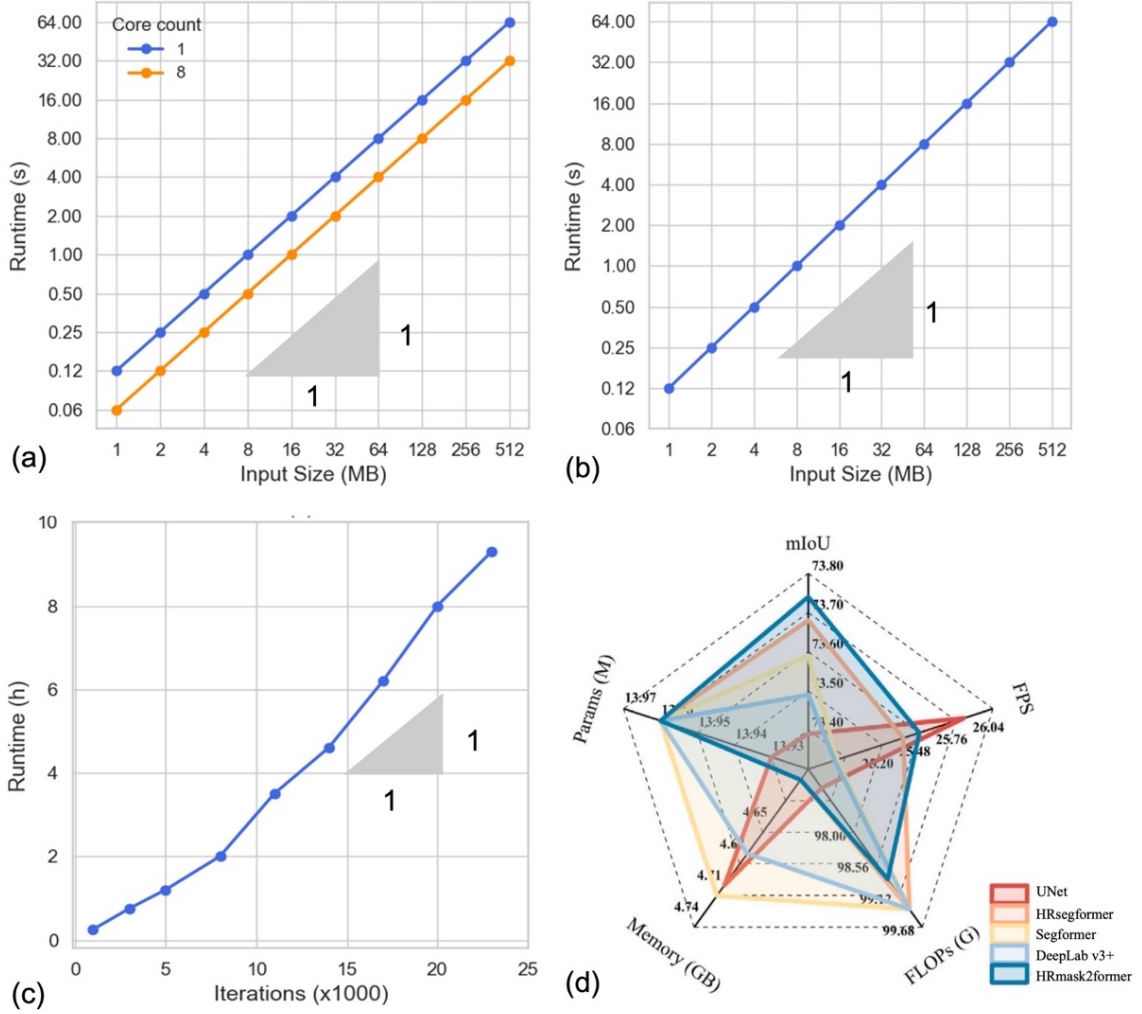


Figure 5: VHR-Seg performance in different tasks. (a) Image dataset splitting performance; (b) postprocessing runtime; (c) iteration runtime. (d) Comprehensive comparison of performance and efficiency for different segmentation models used in VHR-Seg

To evaluate the computational efficiency of VHR-Seg, we carried out profiling of both data processing and model training routines (Fig. 5). Since image splitting and tile merging both iterate once over the $n \times m$ input, their theoretical complexity is $\mathcal{O}(nm)$. With GPU acceleration, the training stage scales proportionally with input size.

In practice, runtime variations had to be addressed. When VHR-Seg divides images with a stride of 0.5, the workload per image quadruples. This overhead is alleviated by parallelizing row-wise operations across CPU cores. Although tile merging also follows $\mathcal{O}(n)$ complexity, the crop-merge strategy significantly speeds up execution, benefiting from vectorized NumPy implementations.

Besides, comprehensive comparison of performance and efficiency for different segmentation models used in VHR-Seg was conducted. The result shows that proposed HRsegformer and

HRmask2former achieve high accuracy while maintaining computational efficiency and inference speed, demonstrating superior overall performance.

3.2 Coarse Stage Semantic Segmentation Performance Using VHR-Seg

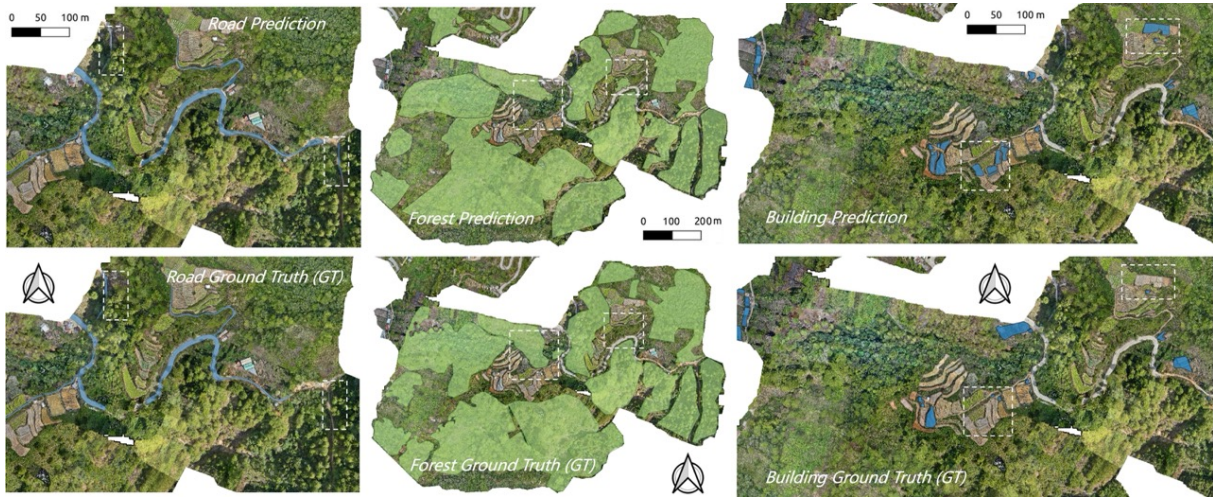


Figure 6: Coarse-stage semantic segmentation results. Predicted maps (top row) are compared with ground truth annotations (bottom row) in the test set. Forest, road, and building are indicated in green, cyan, and blue, respectively.

As shown in Figure 6, the predicted segmentation maps (top row) closely match the ground truth annotations (bottom row). The model accurately distinguishes forest, road, and building regions, which is consistent with the quantitative results (mIoU of 80.35, 74.28, and 71.32, respectively). These results demonstrate that the proposed method achieves reliable coarse-stage semantic segmentation. Moreover, the high-quality predictions obtained at the coarse stage provide a solid foundation for the subsequent fine-grained semantic segmentation stage.

3.3 Fine-grained Stage Semantic Segmentation Performance Using VHR-Seg

In the fine-grained stage semantic segmentation, both **HRsegformer** and **HRmaskformer** trained with VHR-Seg achieved better performance than the baseline models, particularly on small object categories. As shown in Table 1 and Table 2, the proposed models obtained improvements in Palm Tree (best 55.73 in IoU) and Banana Tree (best 60.25 in IoU). These improvements are further supported by the qualitative results in Figure 7, where the proposed models capture tree crowns more completely and reduce fragmented predictions compared to U-Net and DeepLab v3+. For the Car category, which occupies smaller regions in the VHR images, HRsegformer demonstrates competitive performance (75.98 Dice, 65.26 IoU), successfully distinguishing vehicles from surrounding objects. The qualitative examples highlight that cars are more consistently identified by the proposed models, whereas baseline methods often miss or merge them into the background. Both qualitative and quantitative results demonstrate that HRsegformer and HRmaskformer are capable of handling fine-grained vegetation and man-made classes.



Figure 7: Visualization of fine-grained segmentation results on the test dataset. Green indicates forest, light blue indicates roads, and blue indicates buildings. Predicted results (top row) are generated by the proposed model, while the ground truth (bottom row) corresponds to the manual annotations.

Table 1: Experimental results on Fined-segmentation stage The **bold** denotes the best performance among different methods. Δ is boost over U-Net baseline. The evaluation metric is the Dice and mDice.

Method	C-to-F	mDice	Palm Tree	Banana Tree	Road	Car	building	Δ
U-Net [4]	X	63.99	48.13	56.76	71.21	64.72	79.12	-
Segformer w/ b2 [41]	X	66.56	53.67	58.73	74.45	68.49	77.51	-
DeepLab v3+ [6]	X	64.81	51.69	54.72	72.96	65.79	78.89	-
Mask2former [7]	X	70.19	59.39	63.89	77.18	70.29	80.23	-
HRsegformer w/ b0	✓	76.04	68.57	72.19	<u>80.33</u>	75.98	<u>83.13</u>	+12.05
HRmask2former	✓	<u>75.86</u>	<u>65.64</u>	<u>70.71</u>	82.24	<u>74.61</u>	86.10	+11.87

Table 2: Experimental results on Fined-segmentation stage The **bold** denotes the best performance among different methods. Δ is boost over U-Net baseline. The evaluation metric is the IoU and mIoU.

Method	C-to-F	mIoU	Palm Tree	Banana Tree	Road	Car	building	Δ
U-Net [4]	X	51.38	35.26	43.54	60.28	52.35	65.46	-
Segformer w/ b0 [41]	X	54.28	40.52	44.65	63.20	55.63	67.38	-
DeepLab v3+ [6]	X	52.25	37.64	40.57	61.25	52.43	69.34	-
Mask2former [7]	X	58.33	45.34	50.26	66.92	57.84	71.28	-
HRsegformer w/ b2	✓	<u>65.66</u>	55.73	60.25	<u>71.43</u>	65.26	<u>75.64</u>	+14.18
HRmask2former	✓	65.80	<u>52.56</u>	<u>58.37</u>	74.28	<u>63.42</u>	80.35	+14.42

4 Discussion

4.1 VHR-Seg Model Performance

In fine-grained stage, the proposed HRsegformer and HRmask2former achieve notable improvements over baseline models. Benefiting from its specially designed architecture that integrates both high-resolution and low-resolution branches. The low-resolution branch provides global contextual information, and high-resolution branch captures details information. This dual path ensures that semantic consistency is maintained while preserving detailed object boundaries. Thus, HR-series models are capable of getting a better segmentation performance. The ablation study is in Appendix K.

4.2 Effects of on Survey Factors Per-Class Model Performance

As mentioned in section 2.5.1, we have considered three survey factors: object size, Ground Sample Distance (GSD), and training data quantity. Figure M.1 and Figure M.2 illustrate model performance under varying conditions, and detailed GSD reduction scores are provided in Appendix L. The two-stage segmentation results indicate that fine-grained categories are more sensitive to spatial resolution, since these classes usually occupy fewer pixels in imagery and thus require higher GSD to capture sufficient detail for accurate discrimination. In contrast, coarse-level categories typically cover larger areas with richer pixel information. It is much easier to classify the coarse classes than fine class when the GSD is above 0.5m/px. This explains why the performance drop in fine-grained classes is steeper than that of coarse classes as GSD increases. At the coarse stage segmentation, once GSD exceeds 0.5 m/px, performance decreases sharply in forest class. However, due to clear structure, the road and building remain stable in this process.

Except for GSD, the dataset size also has a significant impact on model performance. It will be discussed in the next Section.

4.2.1 Tradeoffs between Dataset Size and DL Performance

Dataset size is an important factor that must also be considered. As shown in Figure M.1 and Figure M.2, both coarse-level categories and fine-grained vegetation classes exhibit a similar trend: model accuracy improves rapidly as the training set increases, but the gain becomes marginal once the dataset exceeds roughly 30% of the total area. This suggests that including about 30% of the area in the training set is sufficient to achieve stable model performance. The goal of training a model is to predict unseen data successfully. Therefore, choosing an appropriate dataset scale is critical to balancing data quantity with diversity when planning large-scale surveys. More broadly, carefully designed dataset sizing strategies not only improve model robustness but also ensure that computational resources are used efficiently, which is essential for scaling up deep learning applications in real-world remote sensing tasks.

4.2.2 Object Size and HR-series Model

Designed to capture fine-grained details, the HR-series model is a powerful architecture can capture fine-grained details in imagery. Combining global attention with scaled attention, it can effectively segment fine-grained classes inside their ROIs. The HR encoder is able to extract detailed information such as textures, and the LR encoder captures contextual features surrounding the fine-grained objects. In this way, the LR branch serves as a teacher, providing guidance to the HR branch for more accurate segmentation. Nevertheless, despite its strong capacity, the HR-series model still relies on the detectability of object size. For example, in

imagery with a GSD of 10 m/px, it is nearly impossible to predict the location of very small objects, such as insects. Thus, object size remains a critical factor in fine-grained semantic segmentation of VHR remote sensing images.

4.3 VHR-Seg is an Efficient Labeling Pipeline



Figure 8: Close-up view showing differences between ground truth road labels (white outlines) and model predictions (green). (1) Both ground truth and predictions were incorrect. (2) Ground truth was incorrect, but the model prediction was correct. (3) The model predicted a road region that was missing from the ground truth.

In our two-stage coarse-to-fine pipeline, an interesting phenomenon was observed in Figure 8 where the model occasionally predicted regions that were not annotated as ground truth, but were semantically correct in reality. This inconsistency comes from various factors, including annotator oversight, fatigue, or limited domain expertise. This observation reveals a common limitation of human-annotated datasets in remote sensing field: even when extensive effort is invested, labeling errors or omissions are difficult to avoid, and these can negatively influence downstream model training. In particular, if such mislabeled regions are directly treated as ground truth, they may introduce misleading training signals that bias the fine segmentation stage and degrade model performance.

The design of VHR-Seg helps solve this problem. The two-stage architecture allows users to conduct a coarse segmentation at first. Regions of interest (ROIs) will be selected after coarse segmentation for further refinement. This step provides an opportunity to detect and correct potential annotation errors before fine-grained segmentation, thereby preventing the propagation of negative labels. This design improves reliability and also reduces the workload for annotation. To elaborate, in coarse stage segmentation, users only need to label general classes, which are not very time-consuming.

At the fine segmentation stage, the ROI-based strategy enables the model to concentrate on subtle intra-class differences without being confounded by inter-class variability. As a result, the model benefits from both a cleaner training signal and a more focused learning objective. Together, these factors enhance both segmentation accuracy and the practical utility of the trained model.

VHR-Seg not only supports more efficient labeling but also actively improves the robustness of semantic segmentation on very high resolution (VHR) remote sensing imagery by mitigating the inherent weaknesses of human-annotated data.

5 Conclusion

This project proposed VHR-Seg, a coarse-to-fine deep learning framework for semantic segmentation in very high resolution remote sensing imagery. Benefiting from this two-stage structure, the framework effectively addressed the challenges of large-scale data and fine-grained object detection. In the coarse stage, broad land-cover categories such as forest, building, and road were reliably segmented. And in the fine segmentation stage, palm trees, banana trees, and cars were detected and predicted by our proposed HR-series model. All experiments were conducted on the real-world Sto. Niño UAV dataset, demonstrating the practical applicability of the framework.

Diverse experiments confirmed that VHR-Seg improved segmentation accuracy. Benefiting from ROIs selection, errors introduced in the coarse stage were corrected, which reduced error accumulation and enhanced overall efficiency. VHR-seg was proved to own a high computational ability in data preprocessing and segmentaion models training by performance analysis. Analyses of ground sampling distance (GSD) and training dataset scale provided valuable insights into the trade-offs between data quality, dataset size, and model performance, highlighting the flexibility of VHR-Seg in adapting to different practical scenarios.

The prediction outputs from VHR-Seg have applications across a wide range of fields, including precision agriculture, biodiversity estimation, and Land Use and Land Cover (LULC) mapping. The framework provides a reproducible and efficient solution for VHR semantic segmentation. Future research may extend this approach to multi-modal data sources such as hyperspectral imagery and LiDAR, or explore domain adaptation strategies to enhance generalization across regions.

6 Acknowledgements

This dissertation represents not only the culmination of my academic journey but also a deeply personal journey of reconciliation with my own history. I want to start by acknowledging myself for the strength and resilience it took to face and come to terms with the parts of my past that are intricately connected to this research topic. This process has been challenging, yet profoundly meaningful, and I am grateful for the courage I found within myself.

I would like to thank my family and friends, whose love and companionship have carried me through the highs and lows of this process. Your loving presence in my life provided the emotional foundation I needed to embark on this journey of self-reconciliation. Thank you.

I am deeply indebted to my supervisor, Dr. Yves Plancherel and Dr. Myriam Prasow-Emond, whose guidance and encouragement have been a constant source of strength throughout this journey. It was truly a privilege to be able to choose this particular topic to research, and I gave her my deepest gratitude for supporting this choice. Her inspiration has not only shaped this research project but also my personal growth. I researched with her unwavering support.

Lastly, I extend my heartfelt thanks to the participants of this study, who generously shared their personal life stories with honesty and courage. Without their contribution and trust, this project would not have been possible. Thank you all.

References

- [1] John E Ball, Derek T Anderson, and Chee Seng Chan. Comprehensive survey of deep learning in remote sensing: theories, tools, and challenges for the community. *Journal of applied remote sensing*, 11(4):042609–042609, 2017.
- [2] Gary Bradski, Adrian Kaehler, et al. Opencv. *Dr. Dobb's journal of software tools*, 3(2), 2000.
- [3] Alexander Buslaev, Vladimir I. Iglovikov, Eugene Khvedchenya, Alex Parinov, Mikhail Druzhinin, and Alexandr A. Kalinin. Albumentations: Fast and flexible image augmentations. *Information*, 11(2), 2020.
- [4] Hu Cao, Yueyue Wang, Joy Chen, Dongsheng Jiang, Xiaopeng Zhang, Qi Tian, and Manning Wang. Swin-unet: Unet-like pure transformer for medical image segmentation. In *European conference on computer vision*, pages 205–218. Springer, 2022.
- [5] Qinglong Cao, Yuntian Chen, Chao Ma, and Xiaokang Yang. Few-shot rotation-invariant aerial image semantic segmentation. *IEEE Transactions on Geoscience and Remote Sensing*, 62:1–13, 2023.
- [6] Liang-Chieh Chen, George Papandreou, Iasonas Kokkinos, Kevin Murphy, and Alan L Yuille. Deeplab: Semantic image segmentation with deep convolutional nets, atrous convolution, and fully connected crfs. *IEEE Trans. Pattern Anal. Mach. Intell.*, 40(4):834–848, 2017.
- [7] Bowen Cheng, Ishan Misra, Alexander G Schwing, Alexander Kirillov, and Rohit Girdhar. Masked-attention mask transformer for universal image segmentation. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 1290–1299, 2022.
- [8] Gong Cheng, Bowei Yan, Peizhen Shi, Ke Li, Xiwen Yao, Lei Guo, and Junwei Han. Prototype-cnn for few-shot object detection in remote sensing images. *IEEE Trans. Geosci. Remote Sens.*, 60:1–10, 2021.
- [9] Seokju Cho, Heeseong Shin, Sunghwan Hong, Anurag Arnab, Paul Hongsuck Seo, and Seungryong Kim. Cat-seg: Cost aggregation for open-vocabulary semantic segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 4113–4123, 2024.
- [10] Megan M Coffey. Balancing privacy rights and the production of high-quality satellite imagery, 2020.
- [11] Henry Cording, Yves Plancherel, and Pablo Brito-Parada. Segmentation of arbitrary features in very high resolution remote sensing imagery. *arXiv preprint arXiv:2412.16046*, 2024.
- [12] Marius Cordts, Mohamed Omran, Sebastian Ramos, Timo Rehfeld, Markus Enzweiler, Rodrigo Benenson, Uwe Franke, Stefan Roth, and Bernt Schiele. The cityscapes dataset for semantic urban scene understanding. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 3213–3223, 2016.
- [13] Daniël Cornelis de Geus. Advances in scene understanding: Towards efficient image segmentation at multiple abstraction levels. 2024.
- [14] Guohui Deng, Zhaocong Wu, Chengjun Wang, Miaozechong Xu, and Yanfei Zhong. Ccanet: Class-constraint coarse-to-fine attentional deep network for subdecimeter aerial image semantic segmentation. *IEEE Trans. Geosci. Remote Sens.*, 60:1–20, 2021.

- [15] Foivos I Diakogiannis, François Waldner, Peter Caccetta, and Chen Wu. Resunet-a: A deep learning framework for semantic segmentation of remotely sensed data. *ISPRS J. Photogramm. Remote Sens.*, 162:94–114, 2020.
- [16] Lei Ding, Jing Zhang, and Lorenzo Bruzzone. Semantic segmentation of large-size vhr remote sensing images using a two-stage multiscale training architecture. *IEEE Transactions on Geoscience and Remote Sensing*, 58(8):5367–5376, 2020.
- [17] Xiaoyi Dong, Jianmin Bao, Dongdong Chen, Weiming Zhang, Nenghai Yu, Lu Yuan, Dong Chen, and Baining Guo. Cswin transformer: A general vision transformer backbone with cross-shaped windows. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 12124–12134, 2022.
- [18] Mark Everingham, Luc Van Gool, Christopher KI Williams, John Winn, and Andrew Zisserman. The pascal visual object classes (voc) challenge. *International journal of computer vision*, 88(2):303–338, 2010.
- [19] Matheus Pinheiro Ferreira, Danilo Roberti Alves De Almeida, Daniel de Almeida Papa, Juliano Baldez Silva Minervino, Hudson Franklin Pessoa Veras, Arthur Formighieri, Caio Alexandre Nascimento Santos, Marcio Aurélio Dantas Ferreira, Evandro Orfano Figueiredo, and Evandro José Linhares Ferreira. Individual tree detection and species classification of amazonian palms using uav images and deep learning. *Forest Ecology and Management*, 475:118397, 2020.
- [20] GA Fotso Kamga, L Bitjoka, T Akram, A Mengue Mbom, S Rameez Naqvi, and Yacine Bouroubi. Advancements in satellite image classification: methodologies, techniques, approaches and applications. *International Journal of Remote Sensing*, 42(20):7662–7722, 2021.
- [21] Danfeng Hong, Bing Zhang, Hao Li, Yuxuan Li, Jing Yao, Chenyu Li, Martin Werner, Jocelyn Chanussot, Alexander Zipf, and Xiao Xiang Zhu. Cross-city matters: A multimodal remote sensing benchmark dataset for cross-city semantic segmentation using high-resolution domain adaptation networks. *Remote Sensing of Environment*, 299:113856, 2023.
- [22] Lukas Hoyer, Dengxin Dai, and Luc Van Gool. Hrda: Context-aware high-resolution domain-adaptive semantic segmentation. In *European conference on computer vision*, pages 372–391. Springer, 2022.
- [23] Teja Kattenborn, Jana Eichel, Susan Wiser, Larry Burrows, Fabian E Fassnacht, and Sebastian Schmidtlein. Convolutional neural networks accurately predict cover fractions of plant species and communities in unmanned aerial vehicle imagery. *Remote Sensing in Ecology and Conservation*, 6(4):472–486, 2020.
- [24] Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alexander C Berg, Wan-Yen Lo, et al. Segment anything. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 4015–4026, 2023.
- [25] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. *Communications of the ACM*, 60(6):84–90, 2017.
- [26] Guosheng Lin, Anton Milan, Chunhua Shen, and Ian Reid. Refinenet: Multi-path refinement networks for high-resolution semantic segmentation. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1925–1934, 2017.

- [27] I Loshchilov. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.
- [28] Bing Lu and Yuhong He. Optimal spatial resolution of unmanned aerial vehicle (uav)-acquired imagery for species classification in a heterogeneous grassland ecosystem. *GI-Science & Remote Sensing*, 55(2):205–220, 2018.
- [29] Lei Ma, Yu Liu, Xueliang Zhang, Yuanxin Ye, Gaofei Yin, and Brian Alan Johnson. Deep learning in remote sensing applications: A meta-analysis and review. *ISPRS journal of photogrammetry and remote sensing*, 152:166–177, 2019.
- [30] Ann E McKellar, Nicholas G Shephard, and Dominique Chabot. Dual visible-thermal camera approach facilitates drone surveys of colonial marshbirds. *Remote Sensing in Ecology and Conservation*, 7(2):214–226, 2021.
- [31] Bipul Neupane, Teerayut Horanont, and Nguyen Duy Hung. Deep learning based banana plant detection and counting using high-resolution red-green-blue (rgb) images collected from unmanned aerial vehicle (uav). *PloS one*, 14(10):e0223906, 2019.
- [32] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khilodov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, et al. Dinov2: Learning robust visual features without supervision. *arXiv preprint arXiv:2304.07193*, 2023.
- [33] QGIS Development Team. *QGIS Geographic Information System*. QGIS Association,
- [34] Nikhila Ravi, Valentin Gabeur, Yuan-Ting Hu, Ronghang Hu, Chaitanya Ryali, Tengyu Ma, Haitham Khedr, Roman Rädle, Chloe Rolland, Laura Gustafson, et al. Sam 2: Segment anything in images and videos. *arXiv preprint arXiv:2408.00714*, 2024.
- [35] Connor Shorten and Taghi M Khoshgoftaar. A survey on image data augmentation for deep learning. *Journal of big data*, 6(1):1–48, 2019.
- [36] Rajendra P Sishodia, Ram L Ray, and Sudhir K Singh. Applications of remote sensing in precision agriculture: A review. *Remote sensing*, 12(19):3136, 2020.
- [37] Tim Heinrich Son, Zack Weedon, Tan Yigitcanlar, Thomas Sanchez, Juan M Corchado, and Rashid Mehmood. Algorithmic urban planning for smart and sustainable development: Systematic review of the literature. *Sustainable Cities and Society*, 94:104562, 2023.
- [38] A Vaswani. Attention is all you need. *Advances in Neural Information Processing Systems*, 2017.
- [39] Libo Wang, Rui Li, Ce Zhang, Shenghui Fang, Chenxi Duan, Xiaoliang Meng, and Peter M Atkinson. Unetformer: A unet-like transformer for efficient semantic segmentation of remote sensing urban scene imagery. *ISPRS Journal of Photogrammetry and Remote Sensing*, 190:196–214, 2022.
- [40] Bin Xie, Jiale Cao, Jin Xie, Fahad Shahbaz Khan, and Yanwei Pang. Sed: A simple encoder-decoder for open-vocabulary semantic segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 3426–3436, 2024.
- [41] Enze Xie, Wenhai Wang, Zhiding Yu, Anima Anandkumar, Jose M Alvarez, and Ping Luo. Segformer: Simple and efficient design for semantic segmentation with transformers. *Advances in neural information processing systems*, 34:12077–12090, 2021.

- [42] Xiaohui Yuan, Jianfang Shi, and Lichuan Gu. A review of deep learning methods for semantic segmentation of remote sensing imagery. *Expert Systems with Applications*, 169:114417, 2021.
- [43] Cheng Zhang, Wanshou Jiang, Yuan Zhang, Wei Wang, Qing Zhao, and Chenjie Wang. Transformer and cnn hybrid deep neural network for semantic segmentation of very-high-resolution remote sensing imagery. *IEEE Transactions on Geoscience and Remote Sensing*, 60:1–20, 2022.
- [44] Liangpei Zhang, Lefei Zhang, and Bo Du. Deep learning for remote sensing data: A technical tutorial on the state of the art. *IEEE Geoscience and remote sensing magazine*, 4(2):22–40, 2016.

A Application of Segment Anything Model (SAM) and DINOv2 to a section of the Sto. Niño dataset

DINOv2 [32] is a visual foundation model trained via self-supervised learning and capable of extracting rich, general-purpose visual features from unlabeled data. SAM [24] is a zero-shot segmentation model that generates masks for arbitrary prompts at interactive speed. Meta has released an online demo, enabling users to segment objects across a large collection of images. A section of the Sto. Niño dataset was used for this comparison. DINOv2 and SAM cannot segment all banana trees and failed to detect palm trees. However, our VHR-Seg successfully segmented all classes of interest. Note that the image is part of the test set; it was not used during model training.

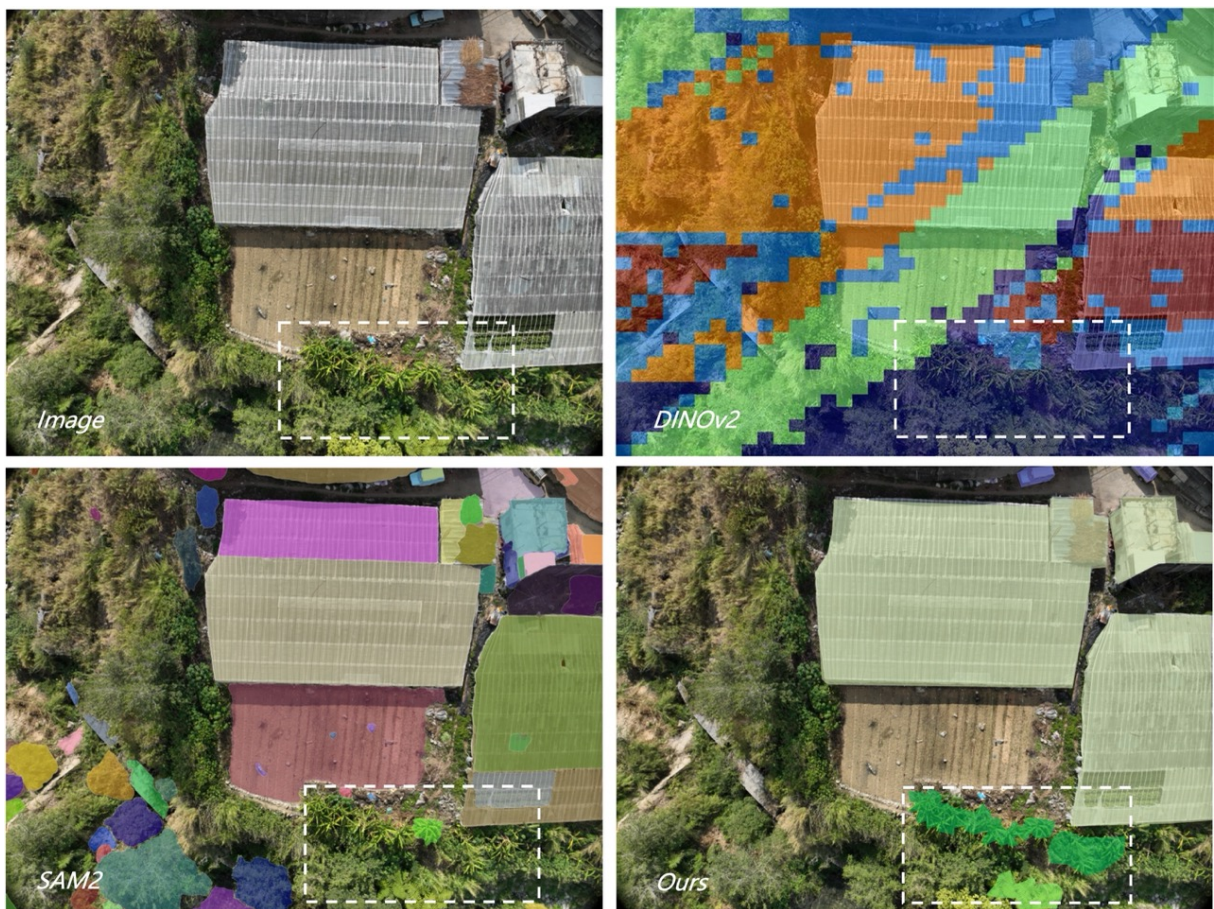


Figure A.1: Application of SAM and DINOv2 to a scene containing banana trees, palm trees, buildings, and cars from a test portion of the Sto. Niño dataset. Clockwise from top left: (1) input image; (2) DINOv2 segmentation output clustered into 5 classes; (3) SAM2 segmentation in automatic mode; (4) segmentation results from our proposed coarse-to-fine system, which successfully identified all target classes.

B Sto. Niño dataset Overview

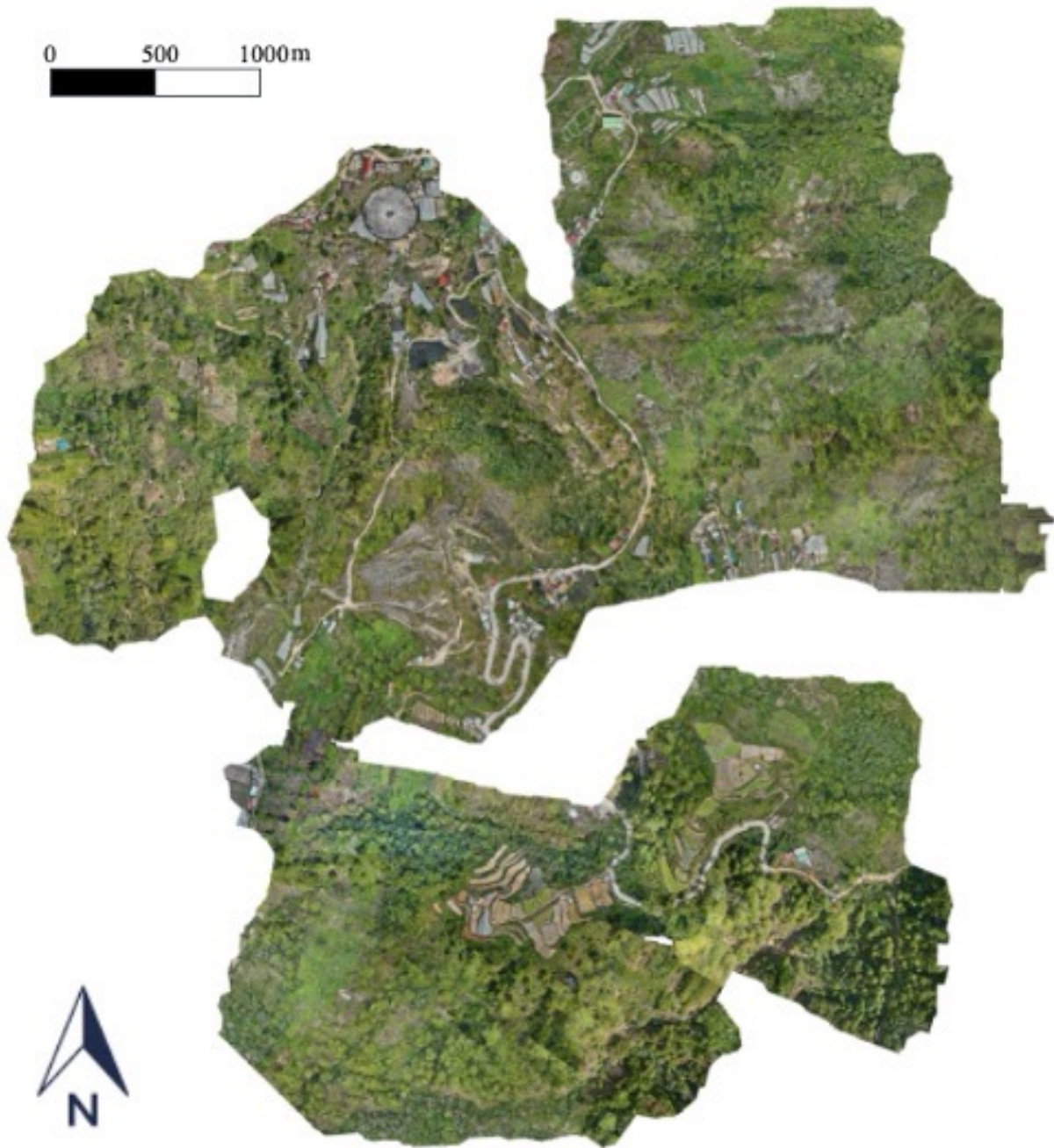


Figure B.1: An overview of the Sto. Niño dataset. The dataset is characterized by its diverse range of vegetation, infrastructure, and varying image quality. The native resolution of around 2cm/px remains fixed over the entire dataset.

C Labeling tutorials for QGIS and CVAT

To support reproducibility, the labeling tutorial documentation provides detailed guidance on the annotation process in QGIS¹. It first gives instructions on installing the required software (QGIS, CVAT, and Docker), followed by step-by-step guidance on importing the dataset and creating image pyramids for efficient navigation. The use of various labeling tools is then demonstrated, and the export process is described to generate segmentation maps in GeoTIFF format. In addition, an alternative tutorial for the CVAT labeling tool is provided, which partially automates label creation. The `label.md` file explains how to generate image tiles with VHR-Seg, upload them to CVAT for labeling, and finally import the generated labels back into VHR-Seg.

D View of Banana trees, Palm trees, and Cars in the Sto. Niño dataset

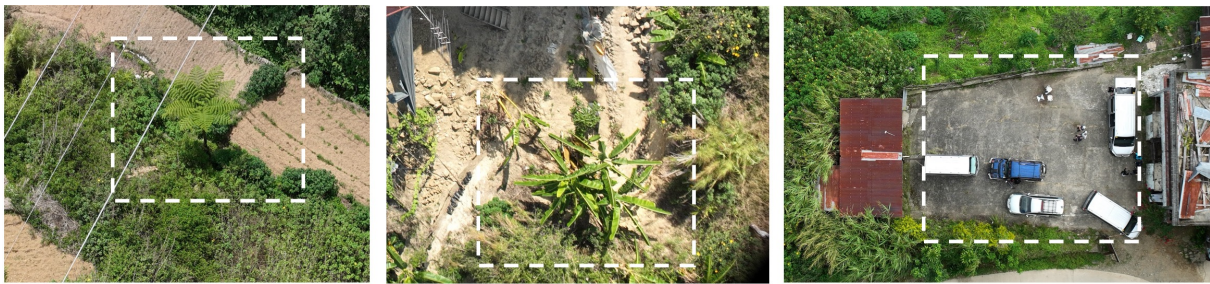


Figure D.1: A partial view of banana trees, palm trees, and cars in the Sto. Niño dataset. The objects of interest are highlighted by dashed boxes for clarity.

E VHR-Seg Task Network

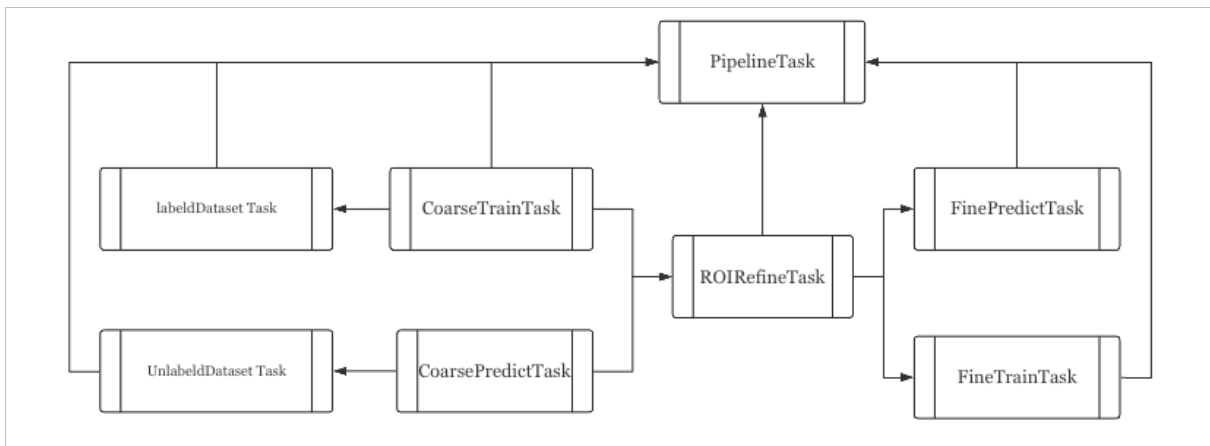


Figure E.1: The VHR-Seg network of Tasks, using UML notation. Arrows indicate dependencies.

¹<https://github.com/ese-ada-lovelace-2024/irp-yc5224>

F Notable features of VHR-Seg

F.1 Parallelized processing

Data processing in VHR-Seg was parallelized using Python's `concurrent.futures` module. The contained `ProcessPoolExecutor` creates a process for each job to run, instead of using threads. VHR-Seg adopts a data pre-chunking strategy, which facilitates effective workload balancing across all processing cores.

F.2 Protection and Hardware compatibility

The long durations tasks may be interrupted due to unexpected resource constraints. VHR-Seg atomically commits the progress of Tasks to a log file, which will be read when restarting. This allows the pipeline to continue computation from the last stable state. VHR-Seg interfaces with the PyTorch training backend to allow training on a wide range of hardware configurations. The code has been tested on CPU-only systems, consumer-grade systems with a single GPU, as well as HPC environments with up to 8 GPUs. Computational requirements for training are adjustable by selecting a particular model variant during model selection, providing trade-offs between training cost and prediction quality.

F.3 Supported data formats and Efficient labeling

The image and label map inputs to VHR-Seg can have any common format, and in particular geospatial formats such as GeoTiff for images and GeoJSON and Shapefiles for label maps are supported. VHR-Seg also supports reading annotations generated with CVAT to help accelerate the labeling procedure.

By employing a coarse-to-fine two-stage segmentation framework, VHR-Seg is able to correct erroneous ground-truth annotations during the coarse stage. Such errors often arise due to differences in annotators' experience, especially in very high-resolution (VHR) RSIs where rich and subtle details increase the likelihood of mislabeling. The hierarchical segmentation design of VHR-Seg provides annotators with the opportunity to revise labels at each stage, thereby reducing error accumulation and preventing the model from learning incorrect feature representations. Moreover, the user interest-guided prediction strategy allows annotators to omit labeling regions that are not of interest, while the region-specific fine model further enhances the accuracy of fine-grained segmentation. And this process decreases the need for labeling.

G Strategies for processing geospatial data in a DL context

G.1 Calculating sample weights for training

Algorithm 1 Calculation of sample weights for the train dataset

Input: Train dataset label tiles *train_dataset_label_tiles*

Output: Sample weights *sample_weights*

```
1 Initialize an empty list of lists class_occurrences;  
2 for each label tile label_tile  $\in$  train_dataset_label_tiles do  
3   Load label_tile as grayscale image m (pixel value = class index);  
4   Count occurrences of each class in m and append to class_occurrences;  
5 end  
6 Compute total_class_occurrences as the sum of lists in class_occurrences;  
7 Initialize empty list sample_weights;  
8 for each sample_class_occurrences  $\in$  class_occurrences do  
9   Initialize empty list s;  
10  for each class i with occurrence class_occurrence do  
11    Append  $\frac{\text{class\_occurrence}}{\text{total\_class\_occurrences}[i]}$  to s;  
12    // Proportion of class i in this tile relative to dataset;  
13  end  
14  Append  $\sum s$  to sample_weights;  
15 end  
16 return sample_weights.
```

G.2 Merging predictions

In the VHR-Seg pipeline, the whole area image is divided into different tiles, which are used as inputs to train or test the model performance. After inference, each image tile has an associated logit tile, where the logits are per-pixel raw scores indicating the model's confidence before normalization (softmax). However, these predictions cannot be directly stitched together into a segmentation map due to overlapping regions between tiles. To address this problem, Researcher [11] developed a merging prediction method, summarized in Algorithm 2.

Algorithm 2 Merging predictions from overlapping tiles

Input: Set of logit tiles *L* overlapping at pixel *p*; Corresponding image tiles *T*; Class set *C*

Output: Predicted class *c_p* for pixel *p*

```
1 for each class c  $\in$  C do  
2   Compute  $m_c = \max_{t \in T} l_t[p][c]$   
3   // Maximum confidence of class c across all overlapping tiles  
4 end  
5 Set  $c_p = \arg \max_{c \in C} m_c$   
6 return cp.
```

H Overview of resolutions

Table H.1: Sto. Niño dataset resolutions and matching platforms

GSD	Resolution (W)	Resolution (H)	Platform
0.08	23662	25228	UAV
0.10	18930	20182	UAV
0.12	15775	16819	UAV
0.15	12620	13455	Maxar (Upscaled)
0.30	6310	6727	Maxar, Digital Globe
0.50	3786	4036	Maxar, Airbus
0.70	2704	2883	Maxar, Planet, CNES, KARI
1	1893	2018	Lockheed Martin Space
3	631	673	Planet
5	379	404	RapidEye Blackbridge
10	189	202	Sentinel-2
15	126	135	Landsat

I Detailed discussion of degradation methods

The degradation methods were illustrated in figure 3. In Method A, both images and their corresponding labels are downsampled before training. This reduces the amount of information input to the model, but maintaining pixel density. Therefore each pixel represents a larger physical distance. The requirements of different model architecture in this process to accommodate for the new input size, making comparisons with baseline model more difficult. Downscaling of labels reduces the precision of gradient computations when updating model weights, which impedes training progress at low resolutions. `INTER_AREA` interpolation of the OpenCV library [2] was used for downscaling.

Method B (Pixelation) does not affect input size, but also reduces the amount of conveyed information, while lowering pixel density. Unlike method A, labels are not degraded, which gives a better estimate of how image quality alone affects predictions. Method B represents a synthetic study of resolution degradation, as multiple pixels in the final tile represent a single pixel in the downsized tile. `INTER_AREA` interpolation was again used for downscaling, followed by bicubic upscaling.

Method C generates "zoomed-out" tiles compared to the original input [2]. Although pixel-level detail is reduced, the model sees a broader spatial context. Pixel density has not changed, so each pixel represents a larger ground area. This simulates resolution changes under higher altitude or lower-resolution sensors. Lower orthomosaic resolution results in fewer training tiles, which may lead to overfitting. All degraded datasets were produced via Lanczos downsampling in QGIS [33].

J HR-series Model design

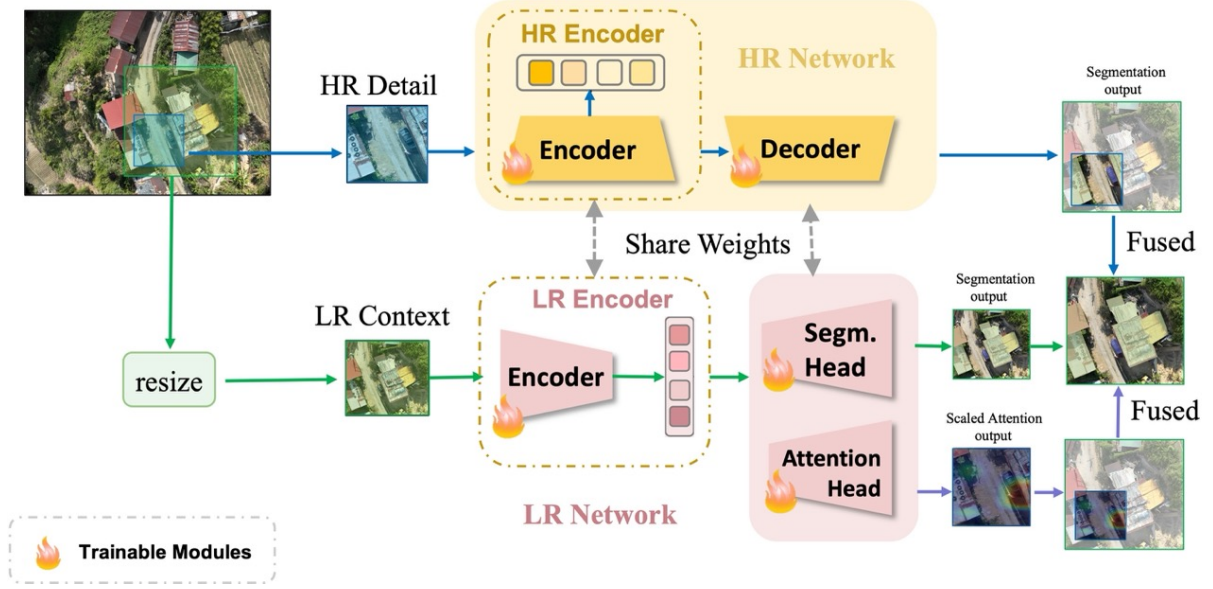


Figure J.1: The HR-series architecture of the proposed model and datasets.

In this section, we describe the HR-series model, which acts as an encoder-decoder module. It is wrapped around the original encoder-decoder architectures of Segformer and Mask2former, resulting in two extended models: HRSegformer and HRMask2former.

HR-series model preliminary. To better utilize the advantages of very high resolution of data, HR model employs high-resolution (HR) crops to capture detailed features while simultaneously using low-resolution (LR) crops to preserve global context. Define x^t is one tile in orthophoto or one image from UAV raw data. $\mathcal{F}$ represents the segmentation model.

LR and HR crop. The combination of multi-level features enables the model to not only focus on the detail features, but also allow the model to make sure it will be gotten in a right context. The context crop x_t^{LR} is cropped from the original HR resolution target domain image x_t^{HR} and bilinear down sampling by the factor s .

$$x_t^{LR} = \zeta(\text{Crop}(x_t^{HR}, \mathbf{b}_L), 1/s), \quad (3)$$

where Crop stands for crop operation, $\mathbf{b}_L = (b_{L,1}, b_{L,2}, b_{L,3}, b_{L,4})$ represents the cropping area, ζ is down sampling operation. We sample the cropping area $\mathbf{b}_{LR}$ in three steps. First, compute the grid interval $k = s \cdot o$, where $o \geq 1$ is the output stride of the segmentation network. Then partition the row and column indices of the high-resolution image into two regular grids with step k ; only these grid points are regarded as valid top-left corners for a crop. From each grid, draw one index with a discrete uniform distribution to obtain the vertical and horizontal start coordinates $(b_{L,1}, b_{L,3})$. To keep the crop inside the image and to fix its size, set the bottom-right corner to $b_{L,2} = b_{L,1} + s h_L$ and $b_{L,4} = b_{L,3} + s w_L$. Consequently, all four edges of the crop fall exactly on the feature-map grid, which guarantees pixel-level correspondence during the subsequent multi-scale fusion stage. The detail crop area is randomly cropped from context crop:

$$x_{t,LR}^{HR} = \text{Crop}(x_t^{LR}, \mathbf{b}_H), \quad (4)$$

where $\mathbf{b}_H = (b_{H,1}, b_{H,2}, b_{H,3}, b_{H,4})$ with a size of $h_H \times w_H$ represents the detail cropping area. And it is randomly cropped from $\mathbf{b}_L$. The context semantic segmentation is $\hat{y}_t^{LR} = \mathcal{F}(x_t^{LR})$

and detail semantic segmentation is $y_{t,LR}^{\hat{H}R} = \mathcal{F}(x_{t,LR}^{HR})$, $\mathcal{F}$ is the semantic segmentation network.

Multi-Resolution Fusion. To balance fine details with global context, we use a learnable scale-attention mechanism to adaptively fuse predictions from the HR detail crop and LR context crop. A attention decoder F_T^A learns to predict the scale attention $a_{LR} = \sigma(F_T^A(F_T^E(x^{LR})))$ to weigh the trustworthiness of LR context and HR detail predictions. The sigmoid function σ ensures a weight in $[0, 1]$, where 1 means a focus on the HR detail crop. The attention is predicted from the context crop as it has a better grasp on the scene layout (larger context). As the predictions are smaller than the inputs due to the output stride o , the crop coordinates are scaled accordingly in the following steps. Outside of the detail crop, the attention is set to 0 as there is no detail prediction available and get a'_{LR} . The HR detail crop is aligned with the (upsampled) context crop by padding it with zeros to a size of $\frac{sh_L}{o} \times \frac{sw_L}{o}$ and get $y_{LR}^{\hat{H}R'}$. The predictions from multiple scales are fused using the attention-weighted sum:

$$\hat{y}_{LR,F} = \zeta((1 - a'_{LR}) \odot \hat{y}_{LR}, s) + \zeta(a'_{LR}, s) \odot y_{LR}^{\hat{H}R'}. \quad (5)$$

The encoder, segmentation head, and attention head are trained with the fused multi-scale prediction and the detail crop prediction. The target loss $\mathcal{L}_{CRAM}^T$ is:

$$\mathcal{L}_{CRAM}^T = (1 - \lambda_d) \mathcal{L}_{ce}(\hat{y}_{LR,F}^T, p_{LR,F}^T, q_{LR,F}^T) + \lambda_d \mathcal{L}_{ce}(\hat{y}_{HR}^T, p_{HR}^T, q_{HR}^T), \quad (6)$$

where $\lambda_d \in [0, 1]$: balance factor between the low-resolution context branch and the high-resolution detail branch. $\mathcal{L}_{ce}(\cdot, \cdot, \tau)$: temperature-scaled cross-entropy, where the temperature τ sharpens ($\tau < 1$) or smooths ($\tau > 1$) the label distribution. $\hat{y}_{LR,F}^T$ is the logits of the scale-attention-fused context prediction after up-sampling to the original image size. $p_{LR,F}^T$ is label map that aligns spatially with $\hat{y}_{LR,F}^T$. $\hat{y}_{HR}^T$ is logits predicted on the high-resolution detail crop and p_{HR}^T is labels for the detail crop. The hyper parameter s and relative crop size will be analysed in Appendix K.1

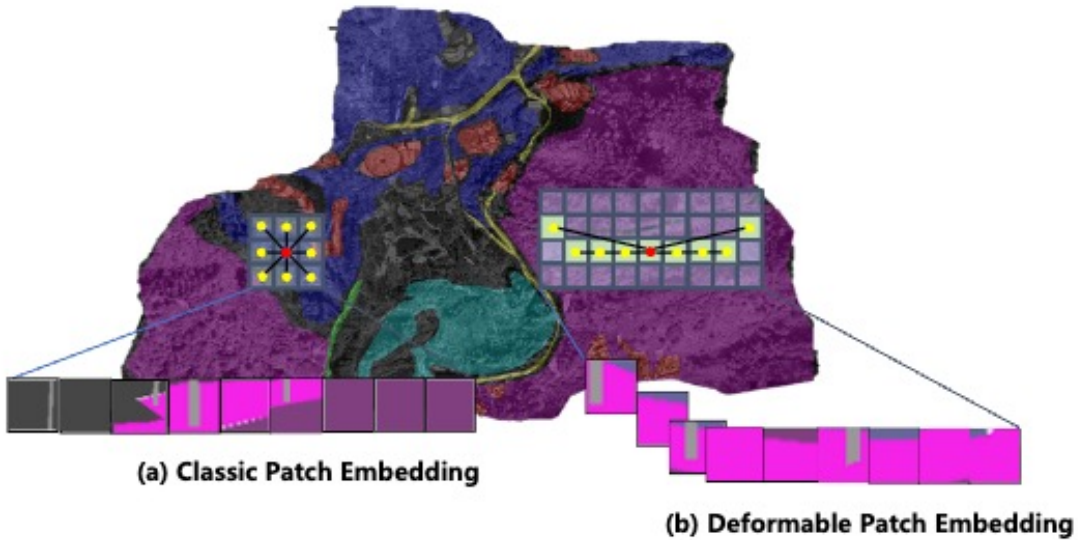


Figure J.2: Visualization of HR-series feature extraction

K Ablation Study

K.1 Analysis of hyper-parameter down sampling factor s and relative crop size

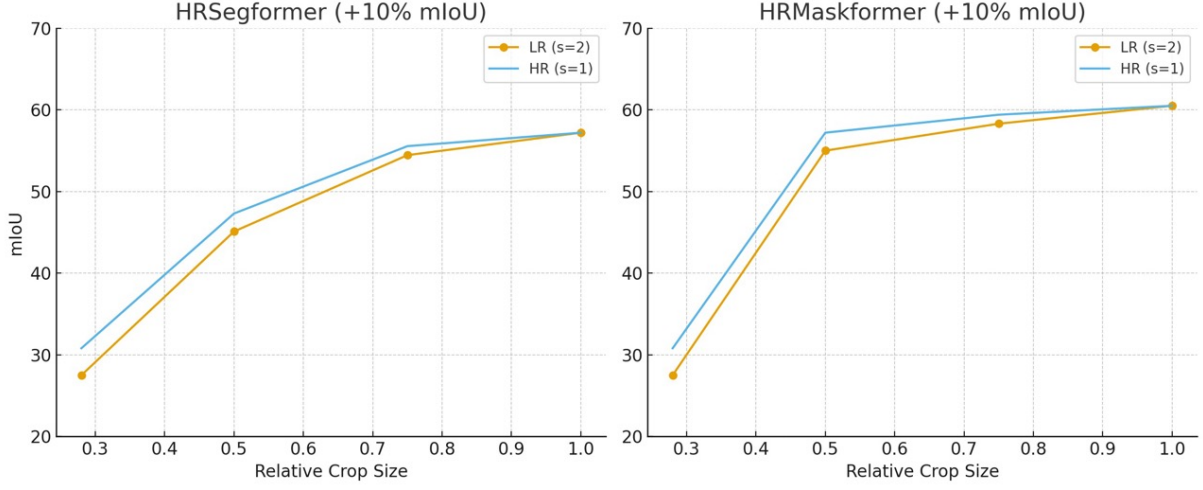


Figure K.1: Segmentation performance over the relative crop size for different resolutions and for both HRsegformer and HRmask2former.

A hyper-parameter analysis focusing on the relative crop size and resolution scale is further conducted. As shown in Figure J.2, both HRSegformer and HRMaskformer consistently benefit from larger crops, where the mIoU improves steeply from 0.3 to 0.75 and then saturates between 0.75 and 1.0. This observation is consistent with our fine-grained stage analysis. Small crops in the early stage encourage the learning of global context, medium crops (0.5–0.75) in the mid stage enhance boundary detection and small-object recognition, while larger crops (≥ 0.75) in the late stage mainly stabilize predictions with diminishing returns.

For the downsampling factor s , high-resolution input ($s = 1$) enables faster convergence and more accurate boundary modeling, whereas low-resolution input ($s = 2$) enhances global invariance and alleviates overfitting in the early phase.

The performances of the HR-series models are also different: HRSegformer shows more balanced gains on vegetation and road-like classes, while HRMaskformer performs better on structured objects such as cars and buildings. In particular, we find that using $s = 2$ with a relative crop size of 0.5 provides a good trade-off in the fine-grained stage, while larger crops combined with HR inputs are more beneficial in the late stage.

L Numeric data on per-class model performance relative to s GSD

Table L.1: Quantative evaluation of coarse stage segmentation performance as GSD is increased

Method	GSD (m/px)	mIoU	mDice	IoU background	IoU building	IoU road	IoU forest	Best val. iteration
n/a	0.08	81.39	82.83	99.64	80.35	74.28	71.32	15000
A	0.10	79.23	81.43	99.08	75.64	71.73	70.48	10000
	0.12	77.51	79.48	99.08	72.38	70.26	68.32	5000
	0.15	74.83	77.21	98.34	65.43	68.32	67.24	2500
	0.30	67.13	70.34	97.6	50.32	55.21	65.40	1250
	0.50	57.33	60.12	96.96	35.24	33.85	63.26	625
	0.70	52.15	55.36	95.76	20.15	30.55	62.15	312
	1	48.18	50.88	96.74	15.33	20.23	60.42	128
	3	38.62	43.16	98.97	5.26	0.00	50.24	10
	5	32.57	33.27	99.64	0.00	0.00	30.65	6
	10	28.10	30.32	99.91	0.00	0.00	12.47	5
	15	26.29	38.33	99.96	0.00	000	5.21	3
B	0.10	78.88	81.70	98.36	76.42	70.86	69.88	13500
	0.12	77.47	78.09	98.34	73.95	69.80	67.80	15000
	0.15	75.74	76.72	97.84	71.10	67.20	66.80	10000
	0.30	67.29	70.11	97.97	51.45	56.32	63.40	9500
	0.50	57.62	64.18	97.11	40.37	32.54	60.45	13000
	0.70	54.39	59.76	96.52	35.34	30.46	55.25	11500
	1	47.30	52.85	96.15	12.75	23.56	56.75	7500
	3	42.08	49.53	94.59	5.60	17.84	50.28	13500
	5	31.35	40.71	92.34	0.00	0.00	33.05	12500
	10	25.42	29.05	88.19	0.00	0.00	13.47	15500
	15	23.07	19.16	88.83	0.00	0.00	3.43	13500
C	0.10	79.48	80.97	98.46	77.33	71.25	70.86	8500
	0.12	79.05	75.48	98.60	78.24	69.98	69.36	6500
	0.15	76.92	72.81	98.34	72.79	68.32	68.23	2500
	0.30	69.71	71.34	97.60	56.04	58.24	66.95	1528
	0.50	53.56	60.12	96.96	25.43	31.56	60.28	630
	0.70	46.40	49.26	95.76	18.73	13.78	57.34	342
	1	41.00	48.88	96.74	4.57	6.32	56.38	90
	3	36.99	38.16	98.97	0.00	0.00	48.75	10
	5	32.48	33.27	99.64	0.00	0.00	30.26	6
	10	28.37	31.32	99.91	0.00	0.00	13.56	5
	15	26.07	29.33	99.96	0.00	0.00	4.32	3

Table L.2: Quantative evaluation of coarse stage segmentation performance as train dataset size is decreased

images in Train Set	mIoU	mDice	IoU Road	IoU forest	IoU building	IoU background
3001	79.25	74.83	71.43	70.48	75.64	99.45
1963	75.9	75.81	69.85	64.23	70.38	99.12
1388	71.58	76.37	66.74	55.24	65.32	99.02
910	66.26	73.81	55.62	50.43	60.24	98.75
254	52.40	67.02	50.47	30.26	30.45	98.45
93	55.31	66.03	32.41	25.42	20.25	98.32
50	44.1	48.94	20.56	18.31	15.31	98.56
23	30.71	34.93	20.35	4.28	0.00	98.21
2	24.55	31.66	0.00	0.00	0.00	98.21
1	24.55	32.39	0.00	0.00	0.00	98.21

Table L.3: Quantative evaluation of fine-grained stage segmentation performance as train dataset size is decreased

images in Train Set	mIoU	mDice	IoU Background	IoU banana tree	IoU palm tree
3001	71.54	74.83	98.64	60.25	55.73
1963	68.56	75.81	98.65	53.21	53.76
1388	60.53	76.37	98.71	40.32	42.56
910	55.10	73.81	98.36	35.44	31.49
254	48.44	67.02	96.93	28.53	19.85
93	12.69	66.03	96.86	17.32	0.00
50	33.59	48.94	95.31	5.46	0.00
23	30.87	34.93	92.61	0.00	0.00
2	28.07	31.66	84.21	0.00	0.00
1	30.85	32.39	92.55	0.00	0.00

Table L.4: Quantative evaluation of fine-grained segmentation performance as GSD is increased

Method	GSD (m/px)	mIoU	mDice	IoU background	IoU banana tree	IoU palm tree	Best val. iteration
n/a	0.08	71.87	73.83	99.64	60.25	55.73	15000
A	0.10	71.48	73.97	98.46	60.25	55.73	10000
	0.12	69.99	73.48	98.60	59.84	51.54	5000
	0.15	66.40	72.81	98.34	55.32	45.53	2500
	0.30	62.75	66.34	97.6	50.33	40.32	1250
	0.50	56.91	60.12	96.96	42.49	31.28	625
	0.70	43.66	49.26	95.76	29.99	5.24	312
	1	38.93	48.88	96.74	18.62	1.45	128
	3	32.99	33.16	98.97	0.00	0.00	10
	5	33.21	33.27	99.64	0.00	0.00	6
	10	33.3	33.32	99.91	0.00	0.00	5
	15	33.32	33.33	99.96	0.00	0.00	3
B	0.10	70.07	72.68	98.36	58.32	53.54	13500
	0.12	69.91	73.09	98.34	59.84	51.54	15000
	0.15	67.18	76.72	97.84	56.32	47.38	10000
	0.30	64.58	70.11	97.97	51.45	44.32	9500
	0.50	56.67	62.18	97.11	40.37	32.53	13000
	0.70	54.11	59.76	96.52	35.34	30.46	11500
	1	35.42	42.85	96.15	5.75	4.36	7500
	3	31.53	33.67	94.59	0.00	0.00	13500
	5	30.78	36.71	92.34	0.00	0.00	12500
	10	29.40	33.05	88.19	0.00	0.00	15500
	15	29.61	34.16	88.83	0.00	0.00	13500
C	0.10	69.62	76.97	98.46	59.84	50.56	8500
	0.12	68.72	73.48	98.60	58.24	49.32	6500
	0.15	66.22	72.81	98.34	52.79	47.53	2500
	0.30	61.93	66.34	97.60	46.04	42.15	1528
	0.50	51.32	60.12	96.96	25.43	31.56	630
	0.70	42.09	49.26	95.76	16.73	13.78	342
	1	37.21	43.88	96.74	8.57	6.32	90
	3	32.99	38.16	98.97	0.00	0.00	10
	5	33.21	33.27	99.64	0.00	0.00	6
	10	33.30	31.32	99.91	0.00	0.00	5
	15	33.32	30.33	99.96	0.00	0.00	3

M Visualization model performance with different survey characteristics

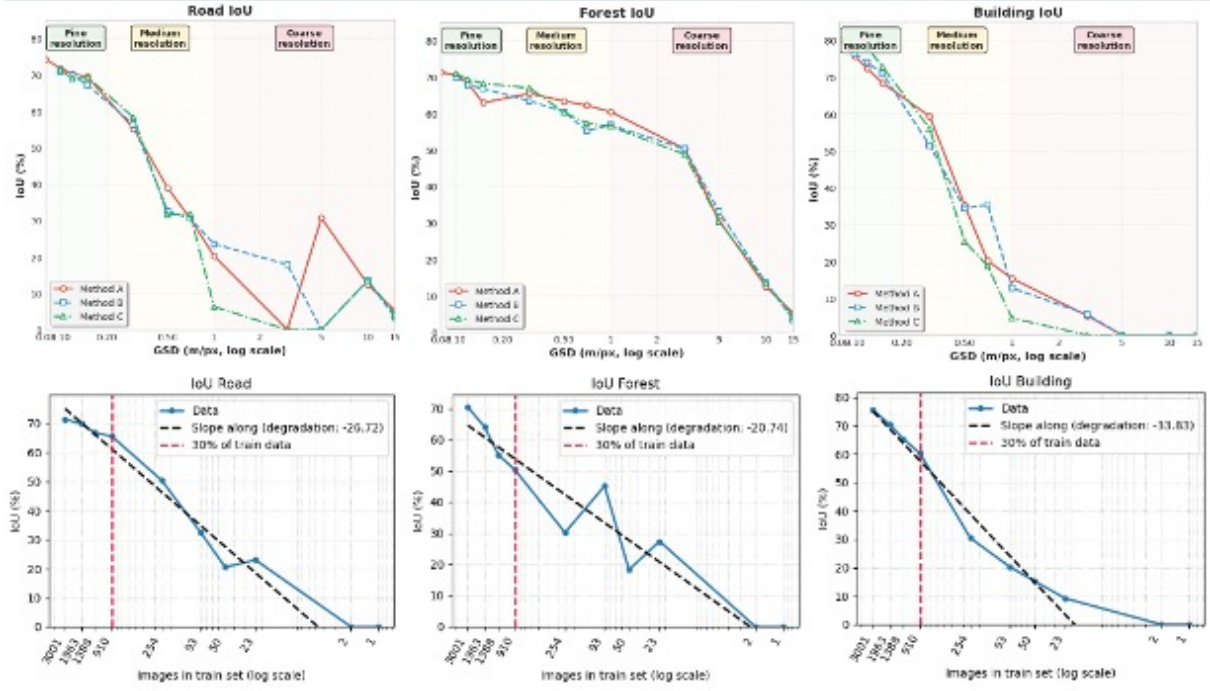


Figure M.1: Model performance visualization under varying survey characteristics in coarse stage. Top row shows the IoU trends for Road, Forest, and Building classes across different sampling scales, while the bottom row presents detailed comparisons of IoU versus the number of training samples.

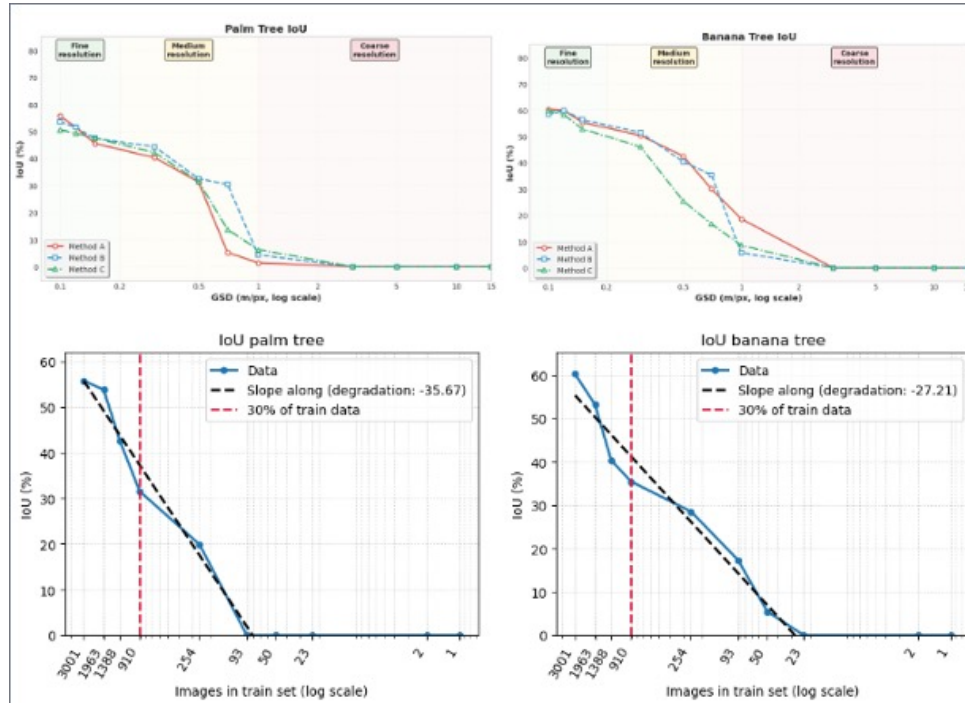


Figure M.2: Model performance on fine-grained vegetation classes in fine-grained stage. Top row shows IoU degradation trends for Palm Tree and Banana Tree under different methods, while bottom row presents IoU versus training set size with slope degradation analysis.

N Variation of Survey Extent with Spatial Resolution

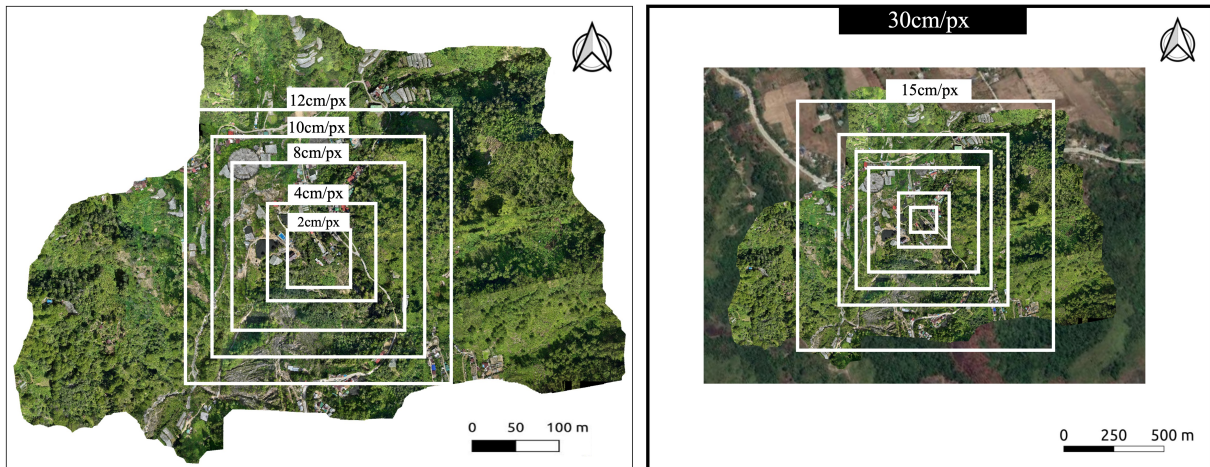


Figure N.1: Variation of survey extent with spatial resolution. Left panel: Sto. Niño dataset overlaid onto satellite imagery, with bounding boxes representing different very high resolution (VHR) ground sampling distances (2–12 cm/px). Right panel: additional extents corresponding to coarser resolutions (15–30 cm/px).

Table N.1: Estimated flight statistics for a DJI Mavic 3M drone imaging each region shown in Fig. N.1, such that the number of pixels in each orthomosaic is the same as for the native dataset with a GSD of 0.022 m/px. Statistics for GSDs of 3, 5, 10, and 15 m/px not shown as the maximum flight altitude of 6km would be exceeded. The maximum legal flight height in most countries is 120 metres, thus a lower quality sensor would need to be fitted on the drone, rather than increasing altitude as shown here. Calculations are based on the specifications provided by DJI.

GSD (m/px)	Altitude (m)	Area to image (km ²)	8h workdays to complete imaging (optimistic estimates)
0.022	75.08	3.06	0.30
0.04	150.16	10.12	1.34
0.08	300.32	40.48	2.22
0.10	375.40	63.26	4.44
0.12	450.48	91.09	5.55
0.15	563.098	142.32	6.66
0.30	1126.20	569.30	8.33
0.50	1876.99	1581.38	16.66
0.70	2627.79	3099.50	27.76
1.00	3753.99	6325.51	38.87